

# INT305 Coursework 1

Name: Ruiyang Wu   Student ID: 2257475  
Programme: Information and Computing Science

October 14, 2025

---

For a training example  $(x_i, y_i)$  with  $K$  classes:  
 $s = Wx_i = \text{score vector } s_j$  ( $s_j =$  predicted score for class  $j$ )

## Task 1: SVM Loss (35')

1) Derive SVM loss  $L_i^{SVM}$  for  $x_i$ ; set  $\Delta$  as the margin hyper-parameter. (1')

### Answer:

For an example  $(x_i, y_i)$ , where  $x_i$  is the feature vector and  $y_i \in \{1, 2, \dots, K\}$  is its class label, we have the score vector  $s_j = Wx_i$ .

We want the score of the right class  $s_{y_i}$  is at least  $\Delta$  (margin) higher than the wrong class  $s_j$ , i.e.  $s_{y_i} \geq s_j + \Delta$ . Therefore, we want:

$$s_j - s_{y_i} + \Delta \leq 0, \forall j \neq y_i \quad (1)$$

In the case that the score of the right class  $s_{y_i}$  is  $\Delta$  (points) higher than the wrong class  $s_j$ , the classifier is correct and receives no penalty, which means:

$$Loss = 0 \quad (2)$$

Otherwise the classifier is incorrect and will be penalized in proportion to how much it violates:

$$Loss = s_j - s_{y_i} + \Delta \quad (3)$$

Thus, by adopting the hinge loss and considering (2)&(3), the Multi-class SVM loss for  $x_i$  is:

$$L_i^{SVM} = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + \Delta) \quad (4)$$

---

2) Derive the gradient  $\frac{\partial L_i^{SVM}}{\partial s_k}$  for

- $k = y_i$  (correct class) (5')
- $k \neq y_i$  (incorrect class) (5')

And explain:

- why is  $\Delta$  only applied to incorrect classes? (3')
- How does  $\Delta$  enforce a “safety margin”? (5')

**Answer:**

Set  $m_j = s_j - s_{y_i} + \Delta$ , we have:

$$L_i^{SVM} = \sum_{j \neq y_i} \max(0, m_j) \quad (5)$$

Let  $g(u) = \max(0, u)$ , the gradient of it (non-differentiable at 0, we take 0 here):

$$g'(u) = \begin{cases} 1, & u > 0 \\ 0, & u \leq 0 \end{cases} \quad (6)$$

**1. In the case  $k = y_i$  (correct class)**

$$\frac{\partial L_i^{SVM}}{\partial s_k} = \frac{\sum_{j \neq y_i} \max(0, m_j)}{\partial s_k} \quad (7)$$

$$\because \text{every } m_{j \neq y_i} \text{ contain } (-s_{y_i}) \quad (8)$$

$$\therefore = \sum_{j \neq y_i} g'(m_j) \cdot (-1) \quad (9)$$

$$= - \sum_{j \neq y_i} 1 \cdot [s_k - s_{y_i} + \Delta > 0] \quad (10)$$

**2. In the case  $k \neq y_i$  (incorrect class)**

$$\frac{\partial L_i^{SVM}}{\partial s_k} = \frac{\sum_{j \neq y_i} \max(0, m_j)}{\partial s_k} \quad (11)$$

$$\because \text{only } m_{j=k} \text{ contains } s_k \quad (12)$$

$$\therefore = g'(m_k) \cdot 1 \quad (13)$$

$$= 1 \cdot [s_k - s_{y_i} + \Delta > 0] \quad (14)$$

**3. Why is  $\Delta$  only applied to incorrect classes:**  $\Delta$  is a hyper-parameter in order to control the difference of the scores between correct and incorrect classes. As I mentioned in (1),  $\Delta$  plays the role of relative gap (or margin) between 2 classes, so there is no need to apply it towards both sides of the equation, or they will be eliminated!

**4. How does  $\Delta$  enforce a “safety margin”:** If there is no  $\Delta$ , i.e.  $\Delta = 0$ , our model only try to maximize the score of correct classifications; when  $\Delta > 0$ , model need to ensure the score of correct class is at least  $\Delta$  scores higher than the incorrect ones. Higher the  $\Delta$  is, wider the “safety margin” is. Making the model more robust against the noise and promotes better generalization performance.

3) Given scores  $s = [3, -1, 4]$  for classes ["cat", "dog", "bird"]; True class  $y_i$  is "cat" (index = 0);  $\Delta = 1$ , compute and explain:

- $L_i^{SVM}$  for each score (3')
- How does  $L_i^{SVM}$  change if  $\Delta = 2$ ? (5')

**Answer:**

1.  $L_i^{SVM}$  for each score:

$$j = 1 \text{ (Score of "dog")} : \max(0, s_1 - s_0 + \Delta) \quad (15)$$

$$= \max(0, -1 - 3 + 1) = \max(0, -3) = 0 \quad (16)$$

$$j = 2 \text{ (Score of "bird")} : \max(0, s_2 - s_0 + \Delta) \quad (17)$$

$$= \max(0, 4 - 3 + 1) = \max(0, 2) = 2 \quad (18)$$

$$\therefore L_i^{SVM} = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + \Delta) \quad (19)$$

$$= 0 + 2 = 2 \quad (20)$$

2. How does  $L_i^{SVM}$  change if  $\Delta = 2$ :

$$j = 1 \text{ (Score of "dog")} : \max(0, s_1 - s_0 + \Delta) \quad (21)$$

$$= \max(0, -1 - 3 + 2) = \max(0, -2) = 0 \quad (22)$$

$$j = 2 \text{ (Score of "bird")} : \max(0, s_2 - s_0 + \Delta) \quad (23)$$

$$= \max(0, 4 - 3 + 2) = \max(0, 3) = 3 \quad (24)$$

$$\therefore L_i^{SVM} = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + \Delta) \quad (25)$$

$$= 0 + 3 = 3 \text{ wow it increased by 1!} \quad (26)$$

4) Using the same score  $s = [3, -1, 4]$  and  $y_i = 0$ :

- Compute  $\frac{\partial L_i^{SVM}}{\partial s}$  (3')
- Interpret the gradient: Why are some values positive, negative, or zero? (5')

**Answer:**

1. Compute:

$$\frac{\partial L_i^{SVM}}{\partial s} = \frac{\partial \sum_{j \neq 0} \max(0, s_j - s_0 + 1)}{\partial s} \quad (27)$$

$$\text{ignore } j = 1, \text{ no active (see (22))} : \quad (28)$$

$$= \left( \frac{\partial \max(0, s_2 - s_0 + 1)}{\partial s_0}, \frac{\partial \max(0, s_2 - s_0 + 1)}{\partial s_1}, \frac{\partial \max(0, s_2 - s_0 + 1)}{\partial s_2} \right) \quad (29)$$

$$= \left( \frac{\partial(s_2 - s_0 + 1)}{\partial s_0}, \frac{\partial(s_2 - s_0 + 1)}{\partial s_1}, \frac{\partial(s_2 - s_0 + 1)}{\partial s_2} \right) \quad (30)$$

$$= (-1, 0, 1) \quad (31)$$

---

**2. Why are some values positive, negative, or zero:** Incorrect class “bird” ( $j = 2$ ) activates the hinge, so its gradient becomes positive, pushing  $s_2$  descend; therefore correct class “cat” ( $j = 0$ ) becomes negative (simply take the derivative), pushing  $s_0$  ascend; “dog” ( $j = 1$ ) did not activate the hinge, so its gradient is zero.

---

## Task 2: Softmax Loss (35')

1) Derive

- Softmax probability  $p_j$  for class  $j$  (1')
- Softmax loss  $L_i^{softmax}$  (1')

**Answer:**

1. Softmax probability  $p_j$  for class  $j$ :

$$p_j = \frac{e^{s_j}}{\sum_{n=1}^K e^{s_n}} \quad (32)$$

2. Softmax loss  $L_i^{softmax}$ :

$$L_i^{softmax} = -\log_e\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right) = -s_{y_i} + \log \sum_j e^{s_j} \quad (33)$$


---

2) Derive the gradient  $\frac{\partial L_i^{softmax}}{\partial s_k}$  for

- $k = y_i$  (correct class) (5')
- $k \neq y_i$  (incorrect class) (5')

And explain:

- Why does minimizing  $L_i^{softmax}$  force  $p_{y_i} \rightarrow 1$ ? (3')

**Answer:**

1. In the case  $k = y_i$  (correct class):

$$\frac{\partial L_i^{softmax}}{\partial s_k} = (-s_{y_i} + \log \sum_j e^{s_j}) \frac{\partial}{\partial s_{y_i}} \quad (34)$$

$$= p_{y_i} - 1 \quad (35)$$

2. In the case  $k \neq y_i$  (incorrect class):

$$\frac{\partial L_i^{softmax}}{\partial s_k} = (-s_{y_i} + \log \sum_j e^{s_j}) \frac{\partial}{\partial s_k} \quad (36)$$

$$= p_k \quad (37)$$

---

**3. Why does minimizing  $L_i^{softmax}$  force  $p_{y_i} \rightarrow 1$ :** From the view of gradient, because  $\forall p_k \in (0, 1)$  s.t.  $\frac{\partial L_i^{sm}}{\partial s_{y_i}} = p_{y_i} - 1 < 0$ ,  $\frac{\partial L_i^{sm}}{\partial s_{k \neq y_i}} = p_k > 0$  Therefore, in the case where there is no constraint during the process of gradient descent,  $p_{y_i}$  will monotonically increase, the limit tends towards 1; meanwhile  $p_k$  will monotonically decrease.

---

**3)** Given scores  $s = [3, -1, 4]$  for classes ["cat", "dog", "bird"]; True class  $y_i$  is "cat" (index = 0), compute and explain:

- $p_j$  (3')
- $L_i^{softmax}$  for the given scores (3')
- What happens to  $L_i^{softmax}$  if all scores are scaled by 2 (i.e.,  $s_{new} = 2s$ ) (4')

**Answer:**

**1. Compute  $p_j$ :**

$$e^{s_0} = e^3 \approx 20.086, e^{s_1} = e^{-1} \approx 0.36788, e^{s_2} = e^4 \approx 54.598, \sum_j e^{s_j} \approx 75.052 \quad (38)$$

$$p_0 = \frac{e^{s_0}}{\sum_j e^{s_j}} \approx \frac{20.086}{75.052} \approx 0.26763, p_1 = \frac{e^{s_1}}{\sum_j e^{s_j}} \approx \frac{0.36788}{75.052} \approx 0.0049, \quad (39)$$

$$p_2 = \frac{e^{s_2}}{\sum_j e^{s_j}} \approx \frac{54.598}{75.052} \approx 0.72747 \quad (40)$$

**2. Compute  $L_i^{softmax}$ :**

$$L_i^{softmax} = -\log_e(p_0) \approx 1.3181 \quad (41)$$

**3. What happens to  $L_i^{softmax}$  if all scores are scaled by 2:  $s_{new} = [6, -2, 8]$**

$$e^6 \approx 403.4, e^{-2} \approx 0.1353, e^8 \approx 2980, \sum_j e^{s_j} \approx 3384 \quad (42)$$

$$p_0 \approx 0.1192, p_1 \approx 0.00004, p_2 \approx 0.8806 \quad (43)$$

$$L_i^{softmax} = -\log_e(p_0) \approx 2.127 \quad (44)$$

When the correct class is not the highest score, scaling up will enhance the relative difference, concentrate the probability to the highest score class (in this case, "bird"), thereby increasing the loss.

In other circumstances, if the correct class got the highest score, scaling up may reduce the loss.

---

**4)** Using the same score  $s = [3, -1, 4]$  and  $y_i = 0$ :

- Compute  $\frac{\partial L_i^{softmax}}{\partial s}$  (3')
- How does the gradient for the correct class differ from SVM? (7')

**Answer:**

**1. Compute:** From Task 2.2:

$$\frac{\partial L_i^{Softmax}}{\partial s_0} = p_0 - 1 \approx -0.7324 \quad (45)$$

$$\frac{\partial L_i^{Softmax}}{\partial s_1} = p_1 \approx 0.0049 \quad (46)$$

$$\frac{\partial L_i^{Softmax}}{\partial s_2} = p_2 \approx 0.7275 \quad (47)$$

$$\therefore \frac{\partial L_i^{Softmax}}{\partial s} \approx (-0.7324, 0.0049, 0.7275) \quad (48)$$

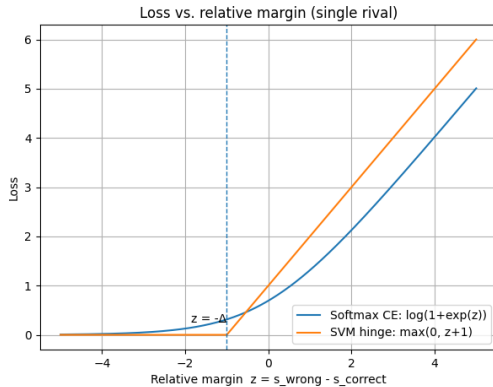
**2. How does the gradient for the correct class differ from SVM:** Softmax provides a soft and probabilistic adjustment, while SVM is harder (0/1).

$$\frac{\partial L_i^{softmax}}{\partial s_{y_i}} = p_{y_i} - 1, \quad \frac{\partial L_i^{SVM}}{\partial s_{y_i}} = - \sum_{j \neq y_i} \mathbf{1}[s_j - s_{y_i} + \Delta > 0] \quad (49)$$

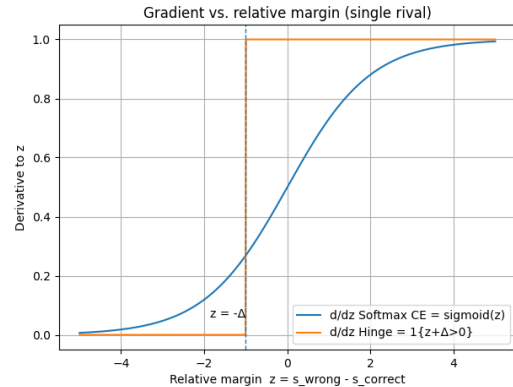
Focus on the correct class, the Softmax gradient changes smoothly: it goes from -1 when  $p_{y_i}$  is small to 0 as  $p_{y_i} \rightarrow 1$ . The higher the probability, the weaker the model push. While the gradient of SVM is discrete constant: it stays negative until all margins are satisfied, then turn to 0.

## Task 3: Comparative Analysis (30')

Before getting into the details, let me use two plots to show the relationship of magnitude and gradient between SVM loss (with  $\Delta = 1$ ) and Softmax loss as the function of relative margin:



(a) Loss vs. Relative margin



(b) Gradient vs. Relative margin

Figure 1: Comparison of  $L_i^{SVM}$  and  $L_i^{Softmax}$

1) Using the same score  $s = [3, -1, 4]$  and  $y_i = 0$ :

- Compare the previous derived losses  $L_i^{SVM}$  and  $L_i^{Softmax}$ , which is larger? (2')
- Why? (3')

**Answer:**

**1. Compare:**  $L_i^{SVM}$  is larger than  $L_i^{Softmax}$ .

$$L_i^{SVM} = 2 \text{ (see (20))} > L_i^{Softmax} \approx 1.3181 \text{ (see (44))} \quad (50)$$

**2. Why:**

In this case, there is 1 incorrect class outside the margin (i.e.,  $s_2 - s_y = 1$ ). As you can see in Fig.1(a). The SVM Loss (Hinge) penalizes this linearly with margin  $\Delta$ :

$$L_i^{SVM} = 1 + \Delta = 2 \quad (51)$$

While the Softmax Loss (Cross Entropy) focus on the allocation of probability ( $p_0 \approx 26.8\%$ ,  $p_1 \approx 0.5\%$ ,  $p_2 \approx 72.7\%$ ), it uses log-sum-exp, which compresses large gaps and spreads penalty across all classes by their probabilities:

$$L_i^{Softmax} = -\log_e \frac{e^3}{e^3 + e^{-1} + e^4} = \underbrace{\log_e \{1 + e^{(-1-3)} + e^{(4-3)}\}}_{\text{grow sublinearly}} \approx 1.318 \quad (52)$$

Therefore, in this case the  $L_i^{SVM}$  is larger than  $L_i^{Softmax}$ .

\* Please notify that when there is little violation (see Fig.1(a)), it is possible for Softmax Loss become higher than SVM Loss, encouraging higher confidence.

**2) Using the gradients from task1 and task2:**

- How does SVM penalize “near-miss” errors (e.g.,  $s_j \approx s_{y_i}$ ) vs. “large-margin” errors? (2')
- How does Softmax adjust probabilities for low-confidence predictions? (3')

**Answer:**

**1. How does SVM penalize:**

As you can see in Fig.1(b), the “Red line” is  $s_{y_i} - \Delta$ . For the “near-miss” errors (e.g.,  $s_j \approx s_{y_i}$ ), once  $s_j > s_{y_i} - \Delta$ , the hinge loss turns on and grows linearly; but for the samples who did not cross the “Red line”, there is no penalty. For the “large-margin” errors, the loss still increases linearly.

In other words, SVM Loss does not scale the gradient with “how badly” a class violates the margin. Every active violator gets the same push, no matter it is “totally wrong” or “nearly right”.

**2. How does Softmax adjust probabilities:**

When the confidence is low ( $0 < p_{y_i} \ll 1$ ), the correct class gets a strong negative gradient ( $(p_{y_i} - 1) \approx -1$ ), strongly lifting  $s_{y_i}$ . Meanwhile, each wrong class get positive gradient with proportional to their probabilities, higher penalty to higher probability class. Finally, as  $p_{y_i} \rightarrow 1$ , gradients decrease smoothly to 0 (see Fig.1(a)).

Hence, Softmax is probability-aware and smooth, allocating larger updates to the most confusing wrong classes.

---

3) Why is SVM loss called “max-margin” and Softmax “cross-entropy”? Please use your own words to define them and show your understanding of the definition and meaning of them. (8’)

**Answer:**

1. Why does SVM loss called “max-margin”:

$$L_i^{SVM} = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + \Delta) \quad (53)$$

We want the score of the right class  $s_{y_i}$  is at least  $\Delta$  higher than the wrong class  $s_j$ , the margin hyper-parameter  $\Delta$  controls the width of the margin. It only care about the class which beyond the margin and gives linear penalty, otherwise the loss is 0.

During the training procedure, SVM Loss trying to minimize the violation (i.e.,  $s_{y_i} \geq s_j + \Delta$ ). Geometrically maximizes the minimum margin between classes: a larger margin ( $\frac{1}{\|W\|}$ ), which makes the model more robust against the noise and promotes better generalization performance. This is why I think SVM loss is called “max-margin”.

2. Why does Softmax loss called “cross-entropy”:

Firstly, let me introduce “cross-entropy”. In the Information Theory, suppose we have 2 distribution:  $q, p$  represent for the target and predicted distribution respectively, the “distance” between two distribution (i.e., cross-entropy) is defined as:

$$H(q, p) = - \sum_k q_k \log_e p_k \quad (54)$$

In the circumstance of supervised classification with one-hot labels, we have  $q_k = 1, k = y_i$ :

$$\Rightarrow H(q, p) = - \log_e p_{y_i} \quad p_{y_i} = \frac{e^{s_{y_i}}}{\sum_{n=1}^K e^{s_n}} = \text{softmax}(s_{y_i}) \quad (55)$$

$$\therefore H(q, p) = - \log(\text{softmax}(s_{y_i})) = L_i^{\text{Softmax}} \quad (56)$$

Clearly, they are the same. What’s more, since  $H(q, p) = \underbrace{H(q)}_{\text{constant}} + D_{KL}(q||p)$ , minimizing the Softmax cross-entropy is equivalent to maximum likelihood estimation (MLE) or minimizing  $D_{KL}(q||p)$ .

---

4) Compare between SVM loss and Softmax loss, which is better and why? (in this case, please not just focus on previous tasks and show your understanding of the two losses) (12’)

**Answer:** I think it depends, the better choice depends on the application goal: Softmax is generally fit for probability modeling and smooth optimization, while explicit geometric margins and sparse updates often prefer SVM loss.

1. **The philosophy of each goal:** Softmax cross-entropy treats logits  $s = (s_1, \dots, s_K)$  as a categorical model with probabilities  $p_k = \frac{e^{s_k}}{\sum_j e^{s_j}}$  and minimizes  $L_i^{\text{Softmax}} = - \log p_y =$



$-s_y + \log \sum_j e^{s_j}$  i.e., maximum likelihood; SVM hinge loss with margin  $\Delta > 0$ ,  $L_i^{\text{svm}} = \sum_{j \neq y} \max\{0, s_j - s_y + \Delta\}$  enforces large margins; with  $\frac{1}{2} \|W\|^2$  regularization it explicitly maximizes geometric separation.

**2. Geometry and numerical shape:** With multiple rivals, softmax grows sublinearly according to their count due to the  $\log \sum e^{z_k}$  compression, while hinge sums violations linearly. Hinge is continuous but non-smooth (at  $z = -\Delta$ ); softmax is smooth everywhere.

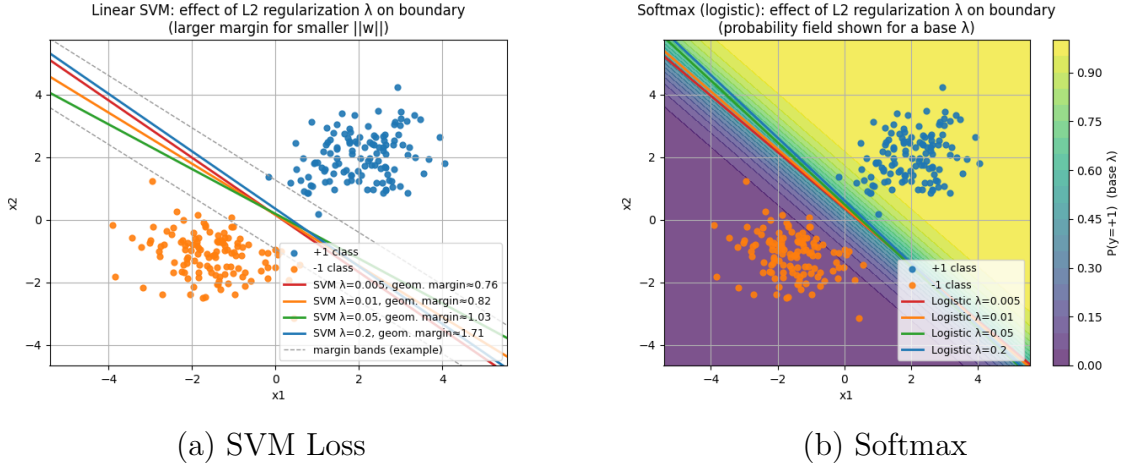


Figure 2: Geometry and numerical shape of  $L_i^{\text{SVM}}$  and  $L_i^{\text{Softmax}}$

### 3. Gradient allocation:

Softmax has  $\frac{\partial L}{\partial s_y} = p_y - 1$ ,  $\frac{\partial L}{\partial s_k} = p_k$  so gradients are dense and confidence-aware: every wrong class gets a push proportional to  $p_k$ , and as  $p_y \rightarrow 1$  all gradients decay smoothly.

Hinge has  $\frac{\partial L}{\partial s_y} = -\sum_{k \neq y} \mathbf{1}[z_k + \Delta > 0]$ ,  $\frac{\partial L}{\partial s_k} = \mathbf{1}[z_k + \Delta > 0]$ , so gradients are sparse and binary: only margin violators are updated (equally), and once safe, a sample gives zero signal.

Softmax thus keeps optimizing the whole distribution; while hinge concentrates on hard points!

**4. Probabilities, entropy, and calibration:** Softmax outputs probabilities and, via cross-entropy, reduces predictive entropy; it works naturally with label smoothing and we can use temperature scaling for calibration. SVM yields margins rather than probabilities.

**5. Robustness against noise:** Softmax loss scales like  $-\log p_y$ ; its gradients are bounded ( $p_y - 1 \in [-1, 0)$ ,  $p_k \in (0, 1)$ ), numerically stable but potentially sensitive to label noise without regularization (can be mitigated by smoothing/ early stopping/ clipping etc.). Hinge grows linearly with violations and is exactly zero in the safe region—sometimes more tolerant to easy examples and certain outliers, but non-smoothness can slow optimization.

**6. Optimization in practice:** Softmax is smooth and well-conditioned in logits, ideal for backprop in deep network. Hinge is continuous but non-smooth; in deep set-

---

tings hinge's sparse/binary updates may converge slower than Cross Entropy's dense, confidence-aware signals.

**Which is better? It depends on the goal.**

Choose Softmax CE if you need well-formed probabilities and strong log-loss performance, or if you are training deep networks where smooth, dense gradients help optimization. It is also preferred when the downstream metrics or decisions depend on calibrated probabilities, likelihood, ranking, or uncertainty estimates, and when stable gradient flow is needed. Use SVM loss when caring about explicit large margins, and want training to focus on hard negatives while ignoring already-safe points.

The key idea is matching the loss to the evaluation target: if you evaluate likelihood/calibration, use Cross Entropy; if you evaluate geometric separation/ margin, use hinge. There is no one single loss fits all. In my experience, practically, during the implementation of machine learning model or deep neural network, we design the loss function that is corresponding to the learning objective and model structure; each part of loss encodes a coherent inductive bias that works when aligned with the problem's objective.

I believe this is the real beauty of machine learning and deep learning: When mathematics resonates with the structures we design, human endow machines with "intelligence". From the innovation of very first data-driven algorithm to the LLMs (Large language models) with hundreds of billions of parameters nowadays, my friend, do not be fear of whether we are the modern "Frankenstein". In the era of artificial intelligence, we are not the only "Homo sapiens"!

—

## Acknowledgments

Thank you for the thoughtful design of the coursework and your careful marking. The questions provided a clear way to examine core ideas in machine learning, and I have done my best to address them.

Wish you a great day.